



Gekkins

ÍNDICE

ÍNDICE	2
1. Introducción:	3
1.1. Motivación y contexto:	3
1.2. Objetivo:	4
1.3. Tecnologías utilizadas:	4
2. Arquitectura general:	7
2.1. Visión general del sistema:	7
2.2. Infraestructura web (front + back):	8
2.3. Arquitectura de la App de escritorio:	9
3. Aplicación de Escritorio:	12
3.1. Estructura del proceso Bun:	12
3.2. Túneles SSH y conexión con Docker:	13
3.3. Worker de estadísticas:	13
3.4. Motor de autoescalado:	14
3.5. Gestión del Swarm:	14
3.6. Bootstrap de nuevos nodos:	15
3.7. Fusión config + Docker en "getNodes":	15
4. Backend Web:	16
4.1. Estructura del proyecto:	16
4.2. Base de datos:	16
4.3. Autenticación:	17
4.4. Rutas y autorización:	17
4.5. Rate limiting:	18
4.6. Despliegue:	18
5. Frontend Web:	19
5.1. Estructura y rutas:	19
5.2. Sistema de autenticación:	19
5.3. Build personalizado:	19
5.4. Loader inicial:	20
5.5. Despliegue en Cloudflare Pages:	20
6. Infraestructura y despliegue:	21
6.1. Visión general:	21
6.2. Cloudflare como capa de red:	21
6.3. Docker Compose multi-entorno:	21
6.4. Gestión de secretos:	22
6.5. Persistencia y arranque automático:	22
7. Seguridad:	23
7.1. Autenticación y gestión de sesiones:	23
7.2. Autorización por roles:	23
7.3. Rate limiting:	23
7.4. Seguridad en la capa de red:	24
7.5. Seguridad en la capa de red:	24
8. Infraestructura Windows:	25
8.1. Topología:	25
8.2. Windows server 2022:	25
8.3. Estructura de OUs:	26
8.4. Trazado de una petición extremo a extremo:	26
9. Conclusiones y futuro:	27
9.1. Conclusiones:	27
9.2. Trabajo futuro:	28

1.Introducción:

1.1. Motivación y contexto:

La infraestructura moderna presenta una paradoja: las herramientas que permiten desplegar y escalar sistemas distribuidos son más potentes que nunca, pero su curva de aprendizaje sigue siendo una barrera real para la mayoría de desarrolladores. Docker, Docker Swarm, la gestión de redes overlay y VPNs, los túneles SSH, el balanceo de carga, el autoescalado o la comunicación multi-nodo son conceptos que, combinados, exigen un perfil técnico muy específico para ser operados con fluidez y quitan tiempo de desarrollo a los llamados “fullstack developers” (como en mi caso).

GEKKINS nace como respuesta a esa brecha. La idea central es convertir la infraestructura distribuida en algo visual, guiado y opinado: que el usuario pueda añadir nodos, conectarlos en red, desplegar servicios y gestionar carga sin necesidad de tocar una terminal ni entender cada capa interna del stack desde el primer momento. No se trata de reinventar la infraestructura, sino de abstraerla — sentarse encima de herramientas como Docker o Kubernetes y convertir su complejidad en una experiencia accesible.

Este proyecto surge también de una motivación personal: la necesidad de integrar en un único trabajo los conocimientos adquiridos a lo largo del ciclo de ASIR y mucho más importante, la necesidad personal de no pelear contra el abismo de configuración de las herramientas modernas cada vez que se quiera escalar o modificar un proyecto. La gestión de sistemas, las redes, la contenedorización, el desarrollo web y la seguridad confluyen en GEKKINS de forma natural, haciendo de él un proyecto que cubre no solo áreas de ASIR sino gran parte de otros ciclos.

1.2. Objetivo:

El objetivo principal del proyecto en su versión actual (v0.1.0) es desarrollar una aplicación de escritorio multiplataforma capaz de gestionar clústeres (multi-nodo) Docker Swarm de forma visual, eliminando la necesidad de interacción directa con la línea de comandos para las operaciones más habituales.

De este objetivo principal se derivan los siguientes objetivos específicos:

- Implementar un sistema de conexión SSH automático a nodos remotos, con soporte para múltiples nodos simultáneos y detección automática del sistema operativo.
- Desarrollar un motor de autoescalado basado en métricas de CPU y RAM en tiempo real, con umbrales configurables por servicio.
- Construir una interfaz de usuario moderna que represente visualmente el estado del clúster, los servicios desplegados y las tareas en ejecución.
- Diseñar e implementar un backend web con autenticación JWT, rate limiting y despliegue en producción mediante Cloudflare Tunnel.
- Desplegar una web pública con CI/CD automático sobre Cloudflare Pages.

El objetivo a largo plazo (v1.0.0) y con potencial de mercado consiste en usar Kubernetes sin que el usuario se de cuenta de nada, abstraer toda la configuración de redes (Vpn, comunicación entre clusters, etc...) y migrar la filosofía de los túneles ssh desde la propia aplicación a una estructura con agentes en cada nodo.

1.3. Tecnologías utilizadas:

TECNOLOGÍA	VERSIÓN	DESCRIPCIÓN
------------	---------	-------------

Aplicación de escritorio

Electrobun	1.18.1	Framework para aplicaciones de escritorio multiplataforma basado en Bun. Alternativa moderna a Electron que usa el WebView nativo del sistema operativo en lugar de Chromium, resultando en ejecutables significativamente más ligeros
Bun	latest	Runtime de JavaScript/TypeScript de alto rendimiento que integra en un único binario

		el motor de ejecución, bundler, gestor de paquetes y cliente de base de datos, entre otros...
React	19.2.6	Biblioteca para construcción de interfaces de usuario basada en componentes. Utilizada tanto para el frontend de la aplicación de escritorio como para la web pública
Tailwind CSS	4.3.0	Framework CSS utility-first que permite construir interfaces directamente en el markup sin escribir CSS personalizado
Framer Motion	12.40.0	Biblioteca de animaciones para React con soporte para transiciones de página, gestos y animaciones físicas basadas en resortes
Dockerode	5.0.0	Cliente oficial de la API de Docker para Node.js/Bun, usado para comunicarse con el daemon de Docker en los nodos del clúster
ssh2	1.17.0	Implementación pura de JavaScript del protocolo SSH2, utilizada para establecer túneles seguros hacia los nodos remotos y ejecutar comandos en ellos
Vite	8.0.14	Bundler y servidor de desarrollo con HMR (Hot Module Replacement) para el frontend de la app de escritorio

Backend Web

Bun	latest	Usado como runtime del servidor, aprovechando su cliente SQL nativo para la conexión con PostgreSQL sin dependencias externas
Hono	4.12.18	Framework HTTP minimalista y de alto rendimiento para Bun/Node.js/Deno. Inspirado en Express pero diseñado para entornos modernos con soporte nativo para TypeScript
PostgreSQL	16	Sistema de gestión de bases de datos relacional. Almacena los usuarios y su información de autenticación

Docker	-	Plataforma de contenedorización usada para desplegar el stack completo (API, base de datos y tunnel) en el servidor de producción
hono-rate-limiter	0.5.3	Middleware de limitación de peticiones por IP para Hono

Frontend Web

React	19.2.6	Misma base que la app de escritorio, reutilizando patrones de componentes y lógica de estado
Tailwind CSS	4.3.0	Misma versión que en la app de escritorio, garantizando consistencia visual entre plataformas
Framer Motion	12.40.0	Animaciones de transición entre páginas y elementos interactivos
Cloudflare Pages	-	Plataforma de despliegue de sitios estáticos con CI/CD automático integrado desde GitHub

Infraestructura y comunicación

Docker Swarm	-	Orquestador de contenedores nativo de Docker para gestión de clústeres multi-nodo, balanceo de carga interno y alta disponibilidad
Cloudflare Tunnel	-	Servicio que permite exponer servicios internos a internet sin abrir puertos en el router ni tener IP pública, estableciendo una conexión saliente cifrada hacia la red de Cloudflare
SSH	-	Protocolo de comunicación segura usado por GEKKINS para conectarse a los nodos del clúster, establecer túneles de red y ejecutar el bootstrap automático de nuevos nodos

2.Arquitectura general:

2.1. Visión general del sistema:

GEKKINS se compone de tres partes diferenciadas que trabajan de forma independiente pero complementaria: la aplicación de escritorio, el backend web y el frontend web.

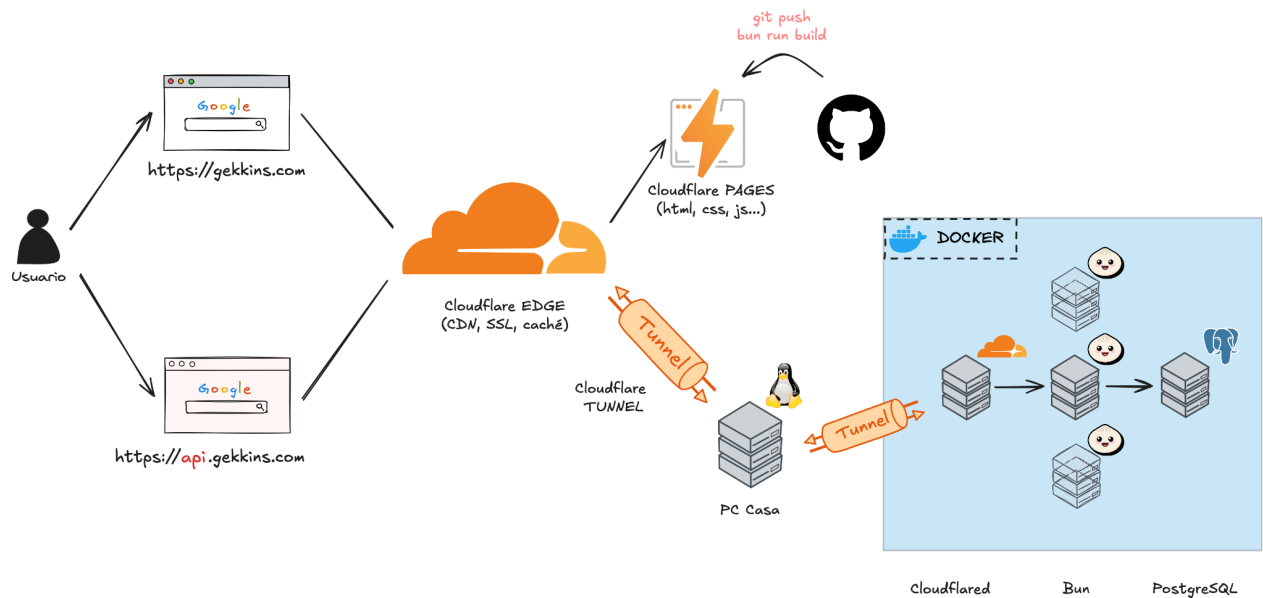
La **aplicación de escritorio** es el núcleo del proyecto. Se ejecuta en el equipo del usuario y se comunica directamente con los nodos del clúster Docker Swarm mediante túneles SSH, sin depender de ningún servidor intermedio. Es la herramienta principal con la que el usuario gestiona su infraestructura.

El **backend web** es una API REST que gestiona la autenticación y los datos de los usuarios registrados en la plataforma. Se despliega en un servidor propio mediante Docker y se expone a internet a través de Cloudflare Tunnel, sin necesidad de IP pública ni apertura de puertos.

El **frontend web** es la página pública de GEKKINS — landing page, registro y login. Se despliega como sitio estático en Cloudflare Pages con CI/CD automático desde GitHub.

Los tres componentes son independientes: la app de escritorio funciona sin conexión al backend web, y el frontend web es simplemente el punto de entrada para nuevos usuarios.

2.2. Infraestructura web (front + back):



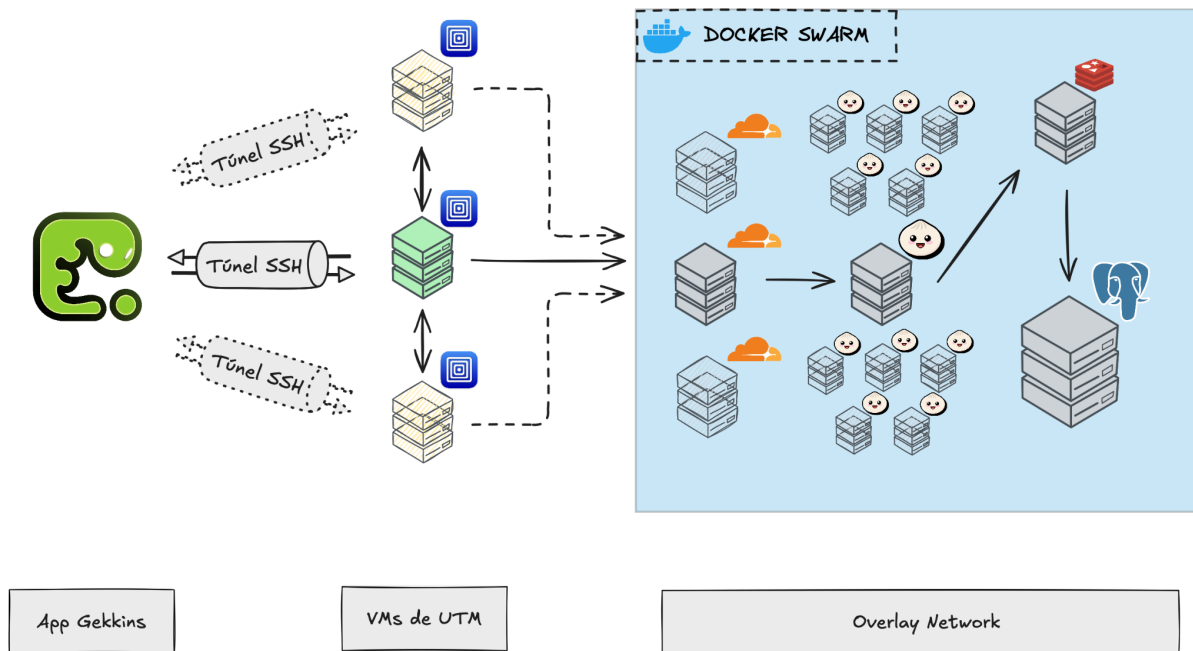
Cuando un usuario accede a **gekkins.com** o **api.gekkins.com**, la petición llega primero al edge de Cloudflare, que se encarga de terminar TLS, aplicar protecciones DDoS y servir el contenido desde la ubicación más cercana al usuario.

Las peticiones a **gekkins.com** se sirven directamente desde Cloudflare Pages — el frontend se despliega como un bundle estático compilado con Bun por el propio Cloudflare y no requiere ningún servidor activo.

Las peticiones a **api.gekkins.com** se enrutan a través de un Cloudflare Tunnel hacia el servidor local (portátil antiguo) donde corre el backend. Este servidor no tiene IP pública ni puertos abiertos al exterior — la conexión la inicia el propio contenedor cloudflared de forma saliente hacia la red de Cloudflare, creando un canal cifrado permanente. Dentro del servidor, cloudflared, api y db corren como tres contenedores en la misma red Docker interna y se comunican por nombre de servicio.

El ciclo de despliegue del frontend es completamente automático: cada git push a la rama principal dispara un build en Cloudflare Pages que compila el proyecto con Bun y despliega el resultado en la CDN global.

2.3. Arquitectura de la App de escritorio:



La aplicación de escritorio sigue una arquitectura de dos procesos separados, característica de los frameworks de escritorio modernos. Por un lado existe el **proceso Bun**, que se ejecuta con acceso completo al sistema operativo y es responsable de toda la lógica de infraestructura: conexiones SSH, comunicación con Docker, autoescalado y gestión de la configuración. Por otro lado existe el **proceso WebView**, que renderiza la interfaz de usuario construida en React y no tiene acceso directo al sistema.

La comunicación entre ambos procesos se realiza a través del sistema RPC de ElectroBun, que permite al WebView invocar funciones del proceso Bun de forma tipada y segura. El contrato entre ambos procesos se define en un único archivo compartido ("src/shared/types.ts"), que actúa como fuente de verdad para ambos lados:

```
export type GekkinsRPC = {
  bun: RPCSchema<
    requests: {
      openExternal: { params: { url: string }; response: void },
      getServices: { params: {}; response: Service[] },
      getTasks: { params: {}; response: Task[] },
      getNodes: { params: {}; response: Node[] },
      getConfig: { params: {}; response: GekkinsConfig },
      setConfig: { params: { config: GekkinsConfig }; response: boolean },
      closeTunnel: { params: {}; response: void },
      scaleService: { params: { serviceId: string, replicas: number }; response: void },
      restartService: { params: { serviceId: string }; response: void },
      toggleAutoScale: { params: { serviceId: string, enabled: boolean }; response: void },
      swarmInit: { params: {}; response: void },
      swarmJoin: { params: { joiningIp: string }; response: void },
      swarmLeave: { params: {}; response: void },
      startBootstrap: { params: { host: string, sshPort: number, username: string, password: string, sshKeyPath: string }; response: void },
      getBootstrapStatus: { params: {}; response: { status: string, error: string | null } },
    }
    messages: {}
  >
  webview: RPCSchema<
    requests: {}
    messages: {}
  >
}
```

- **El problema del timeout RPC y el patrón fire-and-forget**

Electrobun impone un timeout en las llamadas RPC — si el proceso Bun tarda demasiado en responder, la llamada falla. Esto supone un problema para operaciones largas como el bootstrap de un nodo nuevo, que puede tardar varios minutos (*bootstrap = script o conjunto de operaciones para poner a punto algo, en este caso los nodos que se unen al swarm).

La solución adoptada es el patrón fire-and-forget con polling: las operaciones largas se lanzan en segundo plano sin esperar respuesta, y el WebView consulta periódicamente el estado mediante una llamada separada. Por ejemplo, `startBootstrap` inicia el proceso y devuelve inmediatamente, mientras que `getBootstrapStatus` permite al frontend consultar el progreso en intervalos regulares hasta que la operación termina.

- **Conexión con el clúster**

Al arrancar, la aplicación establece un túnel SSH hacia el nodo manager del clúster utilizando la librería `ssh2`. Este túnel redirige el puerto local 2375 al puerto del daemon de Docker en el nodo remoto, permitiendo que Dockerode se conecte a él como si fuera local. Para los nodos worker se establecen túneles adicionales con el mismo mecanismo, permitiendo obtener estadísticas de CPU y RAM de contenedores distribuidos en distintas máquinas.

La aplicación espera activamente a que el puerto 2375 responda antes de considerar el túnel establecido, implementando un sistema de reintentos con `Bun.sleep` entre cada intento.

- Gestión de la configuración

Toda la configuración de la aplicación persiste en `~/.gekkins/config.json`. La estructura principal es `GekkinsConfig`, que almacena el tipo de conexión, la lista de nodos (`connections`), el nombre del stack desplegado, los límites de réplicas y la configuración de autoescalado por servicio. Cada nodo se representa como un objeto `NodeConnection` con su `host`, puerto SSH, usuario, ruta de clave privada, rol en el Swarm y sistema operativo — este último se detecta automáticamente en el primer bootstrap y se guarda para mostrarlo en la interfaz.

```
{
  "connectionType": "remote",
  "socketPath": "/var/run/docker.sock",
  "connections": [
    {
      "label": "manager_1",
      "host": "192.168.64.16",
      "sshPort": 22,
      "username": "guille",
      "sshKeyPath": "/Users/gaffado/.ssh/id_ed25519",
      "os": "linux",
      "role": "manager"
    },
    {
      "label": "worker_1",
      "host": "192.168.64.17",
      "sshPort": 22,
      "username": "guille2",
      "sshKeyPath": "~/.ssh/id_ed25519",
      "os": "linux",
      "role": "worker"
    }
  ],
  "stackName": "test-swarm",
  "maxReplicas": 6,
  "autoScaling": {
    "enabled": false,
    "cpuUpperThreshold": 80,
    "cpuLowerThreshold": 15,
    "intervalSeconds": 5,
    "cycles": 2
  }
}
```

3. Aplicación de Escritorio:

3.1. Estructura del proceso Bun:

El proceso Bun es el núcleo de la aplicación. Al arrancar, inicializa la ventana principal mediante ElectroBun y registra todos los handlers RPC que el WebView puede invocar. El archivo “**rpc_bun.ts**” actúa como punto de entrada de la lógica de negocio, conectando cada método del contrato RPC con su implementación correspondiente:

```
import { BrowserView } from "electrobun/bun"
import type { GekkinsRPC } from "../shared/types.ts"
import { getConfig, setConfig } from "./config.ts"
import { getNodes, getServices, getTasks, closeTunnel, scaleService, restartService, toggleAutoScale, swarmInit, swarmJoin, swarmLeave, startBootstrap, getBootstrapStatus } from "./docker.ts"
import { Utils } from "electrobun"

export const rpc = BrowserView.defineRPC<GekkinsRPC>({
  maxRequestTime: 1200000,
  handlers: {
    requests: {
      openExternal: ({ url }): void => {
        Utils.openExternal(url)
      },
      getServices, getTasks, getNodes, getConfig, setConfig, closeTunnel, scaleService, restartService, toggleAutoScale,
      swarmInit, swarmJoin, swarmLeave, startBootstrap, getBootstrapStatus,
    },
    messages: {}
  }
})
```

El **maxRequestTime** se establece en 20 minutos, aunque ElectroBun impone un timeout interno en el canal de comunicación entre procesos (IPC) que opera de forma independiente. Por ello, operaciones largas como el bootstrap siguen el patrón fire-and-forget en lugar de esperar la respuesta directamente. El **maxRequestTime** es el tiempo máximo que el proceso Bun esperará a que el WebView responda si alguna vez el WebView fuera el que recibe una request del Bun, es decir, para el flujo inverso. En mi caso el flujo es siempre WebView -> Bun.

IPC (Inter-Process Communication) es el mecanismo genérico de comunicación entre procesos del sistema operativo, en este caso, el canal que ElectroBun usa internamente para que el WebView y el proceso Bun se hablen.

RPC (Remote Procedure Call) es la capa que construye ElectroBun encima de ese IPC, la que permite llamar a **getServices()** (por ejemplo) desde el WebView como si fuera una función local.

Al cerrar la aplicación, el proceso intercepta las señales exit, SIGINT, SIGTERM y SIGQUIT para cerrar ordenadamente los túneles SSH activos antes de terminar.

3.2. Túneles SSH y conexión con Docker:

La comunicación con el daemon de Docker en los nodos remotos se realiza a través de **túneles SSH** locales. Para el nodo manager, se establece un túnel que redirige un puerto local al socket de Docker del nodo remoto ("/var/run/docker.sock"), permitiendo que Dockerode se conecte como si el daemon fuera local.

Para los nodos worker se aplica el mismo mecanismo, asignando un puerto local diferente a cada nodo para evitar conflictos. El puerto se deriva dinámicamente de la IP del nodo, por tanto dos máquinas con ip distinta nunca van a tener el mismo puerto:

```
const parts = host.split(".")
const port = 2376 + parseInt(parts[parts.length - 1] ?? "0")
```

Antes de crear cada túnel, se libera el puerto en el caso de que estuviera ocupado por un proceso anterior. Una vez abierto, la aplicación espera activamente a que el daemon de Docker responda en ese puerto antes de considerarlo disponible, con un máximo de 3 reintentos con Bun.sleep(300) entre cada intento.

Todos los túneles y clientes Docker activos se almacenan en mapas (new Map) en memoria (**cacheTunnels**, **cacheDockers**), de forma que las llamadas sucesivas reutilizan la conexión existente sin abrir y cerrar túneles continuamente cada vez que pedimos datos a un nodo.

3.3. Worker de estadísticas:

Para mostrar el uso de CPU y RAM de cada contenedor en tiempo real, el proceso Bun ejecuta un worker independiente cada segundo mediante **setInterval(workerTaskStats, 1000)**. Este worker obtiene la lista de tareas en ejecución del manager, determina en qué nodo corre cada contenedor, se conecta al cliente Docker correspondiente y solicita las estadísticas del contenedor directamente.

El cálculo de CPU sigue la fórmula estándar que dicta la API oficial de Docker:

```
const cpuPercent = systemDelta > 0 ?
  (cpuDelta / systemDelta) * numCpus * 100
  : 0
```

Los resultados se almacenan en un **Map<containerId, ContainerStats>** en memoria. Cuando el WebView solicita las tareas mediante **getTasks**, estas estadísticas se adjuntan a cada tarea directamente desde la caché, sin bloquear la petición con una llamada de red adicional.

3.4. Motor de autoescalado:

El autoescalado funciona por servicio y se activa o desactiva individualmente mediante **toggleAutoScale**. Una vez activado para un servicio, el worker **workerAutoScale** se ejecuta en bucle con el intervalo configurado, calculando la CPU media de todas las tareas del servicio.

El sistema utiliza un mecanismo de **ciclos** para evitar escalados erráticos ante picos puntuales. En lugar de escalar inmediatamente al superar un umbral, incrementa un contador de ciclos consecutivos por encima del umbral superior (cyclesUp) o por debajo del umbral inferior (cyclesLow). Solo cuando ese contador alcanza el número de ciclos configurado se ejecuta el escalado:

```
if (avg_cpu ≥ config.autoScaling.cpuUpperThreshold) {
    cyclesUp.set(serviceId, (cyclesUp.get(serviceId) ?? 0) + 1)
} else if (avg_cpu ≤ config.autoScaling.cpuLowerThreshold) {
    cyclesLow.set(serviceId, (cyclesLow.get(serviceId) ?? 0) + 1)
} else {
    cyclesLow.set(serviceId, 0)
    cyclesUp.set(serviceId, 0)
}
```

El autoescalado hacia abajo nunca reduce el servicio por debajo de una réplica.

3.5. Gestión del Swarm:

La aplicación expone tres operaciones de gestión del clúster:

swarmInit inicializa el Swarm en el nodo manager configurado, estableciendo la dirección de escucha en el puerto estándar 2377. (**todavía no implementado**)

swarmJoin une un nodo worker al Swarm existente. El proceso obtiene el token de join del manager vía Dockerode y lo ejecuta en el nodo worker mediante SSH, evitando joins duplicados comprobando previamente si el nodo ya forma parte del clúster.

swarmLeave desconecta el manager del Swarm de forma ordenada (**todavía no implementado**).

3.6. Bootstrap de nuevos nodos:

El bootstrap automatiza la preparación de un nodo nuevo desde cero — desde una máquina Debian limpia hasta un nodo listo para unirse al Swarm con ssh, docker, permisos correctos, etc.... El proceso se lanza con **startBootstrap** y ejecuta los siguientes pasos de forma secuencial vía SSH: actualización del sistema, instalación de dependencias base, habilitación del servicio SSH, instalación de Docker, configuración del servicio Docker y adición del usuario al grupo docker.

Dado que este proceso puede tardar varios minutos, sigue el patrón fire-and-forget: **startBootstrap** inicia la operación en segundo plano y devuelve una respuesta inmediatamente, mientras que **getBootstrapStatus** permite al frontend consultar el estado hasta que el proceso finaliza o falla, así nada depende del timeout del RPC mencionado antes.

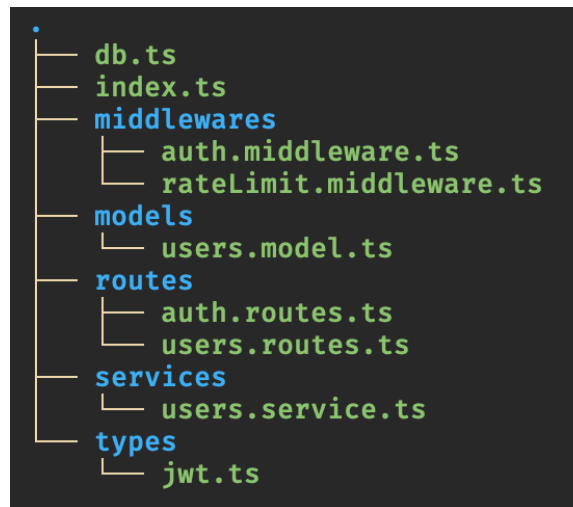
3.7. Fusión config + Docker en “getNodes”:

Una característica destacable de **getNodes** es que combina dos fuentes de información: la configuración local (**config.connections[]**) y el estado real del Swarm reportado por Dockerode. Esto permite mostrar todos los nodos configurados en la UI aunque estén apagados o no sean alcanzables, marcándolos como **offline** en lugar de ocultarlos y que no aparezcan hasta que los detecten los túneles SSH. Adicionalmente, si Docker reporta un sistema operativo diferente al almacenado en la configuración, este se actualiza automáticamente.

4.Backend Web:

4.1. Estructura del proyecto:

El backend sigue una arquitectura en capas con separación clara de responsabilidades. El punto de entrada es `src/index.ts`, que inicializa el servidor Hono, registra el middleware de CORS y monta las rutas. La lógica de negocio se divide en tres capas: rutas (`routes/`), servicios (`services/`) y modelos (`models/`).



4.2. Base de datos:

El esquema de base de datos define una única tabla **users** con un tipo enumerado para el rol:

```
CREATE TYPE user_role AS ENUM ('user', 'admin', 'moderator');
CREATE TABLE IF NOT EXISTS users (
  id SERIAL PRIMARY KEY,
  username VARCHAR(15) UNIQUE NOT NULL,
  password VARCHAR(255) NOT NULL,
  email VARCHAR(40) UNIQUE NOT NULL,
  role user_role NOT NULL DEFAULT 'user',
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT NOW()
);
```

La conexión con PostgreSQL se realiza mediante el cliente SQL nativo de Bun, sin dependencias externas. Esto elimina la necesidad de un ORM o driver adicional, aprovechando la integración directa que ofrece el runtime.

4.3. Autenticación:

El sistema de autenticación se basa en JWT. Al hacer login, el servidor verifica las credenciales del usuario, genera un token firmado con HS256 y lo devuelve junto con los datos básicos del usuario. El token tiene una validez de 24 horas y contiene el **id**, **username** y **role** del usuario en el payload:

```
const payload: JWTPayload = {
  sub: user_valido.id,
  username: user_valido.username,
  role: user_valido.role,
  iat: Math.floor(Date.now() / 1000),
  exp: Math.floor(Date.now() / 1000) + 3600 * 24 // 24 horas desde ahora
}
```

Las contraseñas se almacenan hasheadas mediante **Bun.password.hash**, que usa bcrypt por defecto. La verificación se realiza con **Bun.password.verify**, también integrado en el runtime sin dependencias adicionales.

4.4. Rutas y autorización:

La API expone los siguientes endpoints:

Método	Ruta	Acceso	Descripción
POST	/api/users	Público	Registro de nuevo usuario
POST	/api/auth/login	Público	Login y obtención de JWT
PATCH	/api/users/:id	JWT	Actualización de usuario propio o por admin
DELETE	/api/users/:id	JWT + admin/moderator	Eliminación de usuario
GET	/api/users/:id	JWT + admin/moderator	Consulta de usuario por id
GET	/api/users	JWT + admin/moderator	Listado de todos los usuarios

La autorización se implementa en dos niveles. El middleware **authMiddleware** verifica el JWT en cada ruta protegida. Dentro de cada handler, se comprueba el rol del usuario extraído del **payload** para determinar si tiene permisos sobre la operación concreta.

Una consideración de seguridad importante es el filtrado de campos en las operaciones de actualización. Aunque el servicio define el tipo **Partial<Pick<Users, 'username' | 'email' | 'password'>>**, este tipado solo existe en compilación. En runtime, el body del request puede contener cualquier campo — incluido **role**. Por ello, el handler extrae explícitamente sólo los campos permitidos antes de pasarlos al servicio:

```
// NO dejo pasar "role" porque si no alguien se puede cambiar a "admin"
const data: Partial<Pick<Users, 'username' | 'email' | 'password'>> = {}
if (body.username) data.username = body.username
if (body.email) data.email = body.email
if (body.password) data.password = body.password
```

4.5. Rate limiting:

El rate limiting se implementa mediante **hono-rate-limiter**, con configuraciones diferenciadas por tipo de operación. El endpoint de login tiene un límite de 10 peticiones cada 15 minutos por IP para mitigar ataques de fuerza bruta. Las rutas de gestión de usuarios tienen un límite más estricto de 5 peticiones por hora, adecuado para operaciones administrativas poco frecuentes.

4.6. Despliegue:

El backend se despliega en un servidor Debian mediante Docker Compose con tres contenedores: **api**, **db** y **cloudflared**. Los contenedores se comunican por red interna Docker sin exponer puertos al exterior. El acceso desde internet llega exclusivamente a través del Cloudflare Tunnel, que enruta las peticiones a **api.gekkins.com** hacia el contenedor api en **http://api:3000**.

Se utilizan dos archivos compose: **docker-compose.yml** con la configuración base (para testeo en localhost) y **docker-compose.prod.yml** con la configuración específica de producción, que se combinan en el arranque:

```
> docker compose --env-file .env.production \
-f docker-compose.yml \
-f docker-compose.prod.yml up -d
```

5.Frontend Web:

5.1. Estructura y rutas:

El frontend es una SPA construida con React y compilada con un build script basado en Bun, sin Vite ni Create React App. La aplicación define cuatro rutas principales gestionadas por React Router:

Ruta	Página	Acceso
/	Home	Público
/login	Login	Público
/register	Register	Público
/dashboard	Dashboard	Autenticado

Las transiciones entre páginas se gestionan con **AnimatePresence** de Framer Motion en modo **wait**, lo que garantiza que la página saliente completa su animación de salida antes de que entre la siguiente.

5.2. Sistema de autenticación:

La autenticación se gestiona mediante un contexto React (**AuthProvider**) que expone el token JWT y una función para actualizarlo a cualquier componente de la aplicación. El token se inicializa leyendo **localStorage** al montar el provider, de forma que la sesión persiste entre recargas:

```
const [token, setToken] = useState<string | null>(localStorage.getItem('token'))
```

Las páginas protegidas comprueban la presencia del token mediante el hook **useAuth** y redirigen al login si no existe. Las páginas de login y registro hacen lo contrario — si ya hay token activo, redirigen al dashboard.

5.3. Build personalizado:

En lugar de usar un bundler estándar como Vite, el frontend utiliza un script **build.ts** propio que invoca la API de **Bun.build** directamente. El script escanea

automáticamente todos los archivos HTML del directorio **src**, los toma como entypoints y genera el bundle en **dist/** con minificación activada.

La característica más relevante del build es la inyección de variables de entorno en tiempo de compilación mediante el campo **define**:

```
define: {  
  "process.env.NODE_ENV": JSON.stringify("production"),  
  "import.meta.env.API_URL": JSON.stringify(process.env.API_URL ?? ""),  
},
```

Esto permite que la URL de la API sea diferente en local y en producción sin modificar el código. En producción, Cloudflare Pages inyecta la variable **API_URL** como variable de entorno del proceso de build, y el bundler la sustituye directamente en el bundle resultante. El archivo **src/config.ts** la exporta para uso en toda la aplicación:

```
export const API_URL = import.meta.env.API_URL ?? ""
```

Un detalle importante: el optional chaining (**import.meta.env?.API_URL**) impide que Bun reconozca el patrón en el AST y aplique la sustitución. La variable debe referenciarse sin **?** para que el **define** funcione correctamente.

5.4. Loader inicial:

Al primer arranque de la aplicación se muestra un loader animado durante 1,9 segundos. El comportamiento está controlado por una variable de módulo (**loaderMostrado**) que garantiza que el loader solo se muestre una vez por sesión, incluso si el componente **App** se remonta. La Navbar y el contenido principal permanecen ocultos hasta que el loader completa su animación.

5.5. Despliegue en Cloudflare Pages:

El frontend se despliega automáticamente en Cloudflare Pages. Cada push a la rama principal del repositorio dispara un build en la infraestructura de Cloudflare, que ejecuta el comando **bun run build.ts** y publica el contenido de **dist/** en la CDN global. El dominio **gekkins.com** apunta directamente a Pages con HTTPS forzado y TLS 1.2 mínimo para evitar vulnerabilidades.

6. Infraestructura y despliegue:

6.1. Visión general:

El sistema completo se distribuye en tres entornos de ejecución independientes: la CDN de Cloudflare para el frontend estático, un servidor Debian doméstico para el backend, y el equipo del usuario para la aplicación de escritorio. Ninguno de estos entornos requiere una IP pública propia — Cloudflare actúa como intermediario entre internet y los servicios internos.

6.2. Cloudflare como capa de red:

Cloudflare gestiona tres aspectos críticos de la infraestructura:

Edge y CDN = Todo el tráfico hacia **gekkins.com** y **api.gekkins.com** pasa primero por el edge de Cloudflare, que termina TLS, aplica protecciones DDoS y sirve el contenido estático (no aplicable a **api.gekkins.com**) desde la ubicación más cercana al usuario.

Pages = El frontend se sirve directamente desde la CDN de Cloudflare como contenido estático. No existe ningún servidor web detrás — el bundle compilado por Bun se sube directamente a la infraestructura de Pages en cada deploy.

Tunnel = En lugar de abrir puertos en el router o contratar una IP fija, el backend establece una conexión saliente permanente hacia la red de Cloudflare mediante el contenedor **cloudflared**. Cloudflare enruta las peticiones a **api.gekkins.com** a través de ese túnel hasta el contenedor **api**.

6.3. Docker Compose multi-entorno:

El backend se gestiona con una estructura de **tres** archivos compose que se combinan según el entorno:

docker-compose.yml define la configuración base compartida: la imagen de PostgreSQL, los volúmenes persistentes y la inicialización del esquema mediante **db/init.sql**.

docker-compose.override.yml se aplica automáticamente en local al ejecutar `docker compose up`. Añade el servicio `api` con `hot reload`, expone el puerto 3000 al host y el 5432 de PostgreSQL para conectarse con herramientas como DBeaver.

docker-compose.prod.yml se aplica manualmente en producción. Añade el servicio `cloudflared`, elimina los puertos expuestos al host y configura el comando de arranque sin `hot reload`.

El arranque en producción combina los dos archivos explícitamente:

```
> docker compose --env-file .env.production \
-f docker-compose.yml \
-f docker-compose.prod.yml up -d
```

6.4. Gestión de secretos:

Los secretos se gestionan mediante archivos **.env** que nunca se suben al repositorio. El entorno local usa **.env** con credenciales de desarrollo, y el servidor de producción tiene **.env.production**. Este archivo contiene las credenciales reales de PostgreSQL, el JWT secret generado con `openssl rand -hex 32` y el token del Cloudflare Tunnel.

Las variables de entorno del frontend se gestionan de forma diferente — la única variable necesaria (**API_URL**) se configura en el panel de Cloudflare Pages y se puede inyectar en el bundle durante el build o con un **.env.production** también.

6.5. Persistencia y arranque automático:

Los datos de PostgreSQL se almacenan en un volumen Docker (**db_data**), que persiste independientemente del ciclo de vida de los contenedores. Esto permite actualizar o reiniciar el servicio **api** sin afectar a los datos.

Para garantizar que el backend arranca automáticamente tras un reinicio del servidor, el servicio Docker está habilitado mediante **systemctl enable docker**.

7.Seguridad:

7.1. Autenticación y gestión de sesiones:

El acceso a los endpoints protegidos requiere un JWT válido firmado con el algoritmo HS256. El token se genera en el servidor al hacer login y tiene una validez de 24 horas. El secret de firma se genera con **openssl rand -hex 32** y se almacena únicamente en el servidor de producción — nunca en el repositorio.

Las contraseñas se almacenan en la base de datos hasheadas con bcrypt mediante **Bun.password.hash**. bcrypt es un algoritmo de hashing lento por diseño, lo que dificulta los ataques de fuerza bruta sobre la base de datos en caso de filtración.

7.2. Autorización por roles:

El sistema implementa tres roles: **user**, **moderator** y **admin**. Cada endpoint protegido comprueba el rol extraído del payload del JWT antes de ejecutar la operación:

- Un usuario con rol **user** solo puede modificar su propio perfil
- Un usuario con rol **moderator** puede consultar y eliminar cualquier usuario.
- Un usuario con rol **admin** tiene acceso completo, incluyendo modificar cualquier perfil.

Esta comprobación se realiza en el handler de cada ruta, no en un middleware genérico, lo que permite un control granular por operación.

Aunque el servicio define los tipos correctos en TypeScript, el body de un request HTTP es **any** en runtime. Un usuario malintencionado podría enviar **{ "role": "admin" }** en un **PATCH** para escalar sus propios privilegios. Para evitarlo, el handler extrae explícitamente solo los campos permitidos antes de pasarlos al servicio, ignorando cualquier campo adicional que pueda venir en el body.

7.3. Rate limiting:

Se aplica rate limiting por IP en todas las rutas de la API para mitigar ataques automatizados. El endpoint de login tiene un límite de 10 peticiones cada 15

minutos, reduciendo la viabilidad de ataques de fuerza bruta sobre credenciales. Las rutas de gestión de usuarios tienen un límite de 5 peticiones por hora, adecuado para operaciones administrativas.

7.4. Seguridad en la capa de red:

El backend no expone ningún puerto directamente a internet. Todo el tráfico entra a través del Cloudflare Tunnel, que establece una conexión saliente cifrada desde el servidor hacia Cloudflare. Esto elimina la superficie de ataque habitual asociada a puertos abiertos y proporciona las protecciones de la red de Cloudflare como capa adicional.

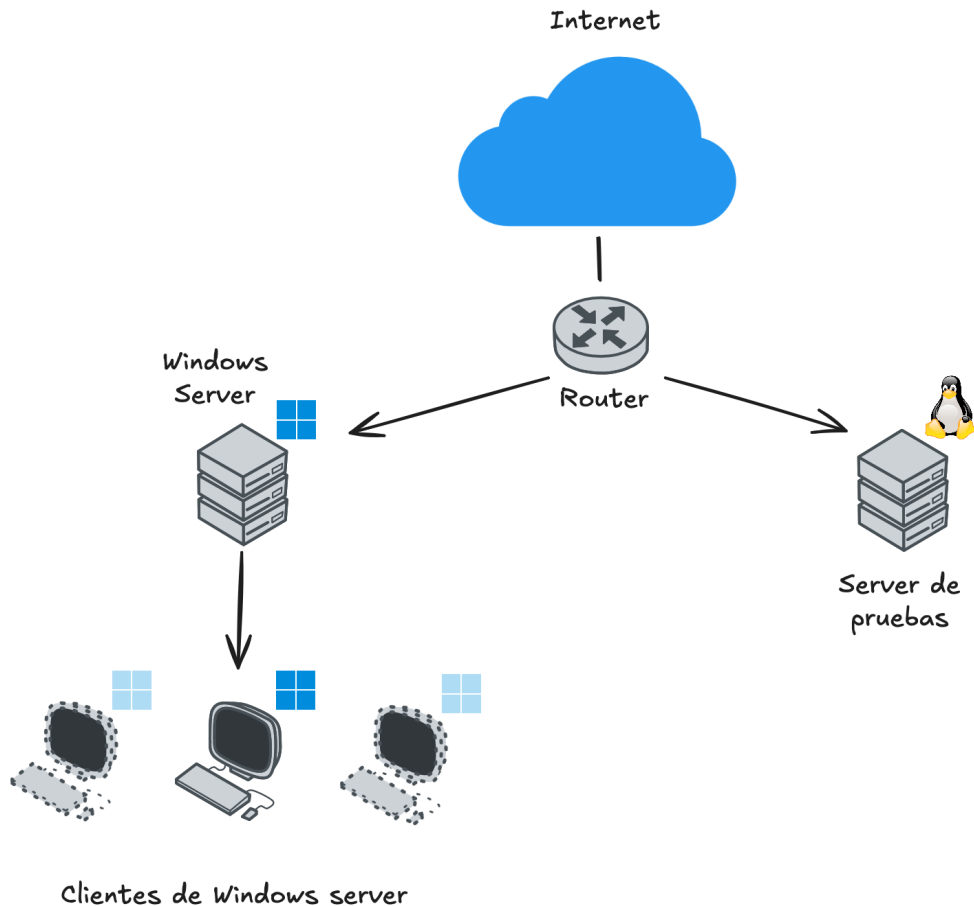
La comunicación entre GEKKINS y los nodos del clúster se realiza exclusivamente mediante SSH con autenticación por clave pública, sin contraseñas. Las claves privadas se almacenan localmente en el equipo del usuario y nunca se transmiten.

7.5. Seguridad en la capa de red:

El servidor configura CORS explícitamente, permitiendo únicamente los orígenes conocidos de la aplicación. Cualquier petición desde un origen no autorizado es rechazada antes de llegar a los handlers. Ahora mismo se permite el acceso desde: **['https://gekkins.com', 'http://localhost:3000', 'http://localhost:3001', 'http://localhost:5173', 'http://localhost:50000']**, los localhost son para desarrollo.

8. Infraestructura Windows:

8.1. Topología:



La infraestructura se divide en dos redes internas interconectadas por un router Ubuntu. La red **red-empresa** (192.168.1.0/24) alberga el Windows Server y el cliente Windows. La red **red-servidores** (192.168.2.0/24) alberga el servidor de aplicaciones con Docker. El dominio utilizado es **gekkins.local**.

8.2. Windows server 2022:

El servidor (**192.168.1.10**) tiene instalados los roles de Active Directory, DNS, DHCP y FTP. El cliente Windows obtiene su configuración de red automáticamente por DHCP (rango **192.168.1.100-200**, gateway **192.168.1.1**, DNS **192.168.1.10**) y está unido al dominio **gekkins.local**, desde donde los usuarios se autentican con sus credenciales de Active Directory.

8.3. Estructura de OUs:

Se han creado cuatro OUs reflejando los departamentos de la startup:

Frontend-Devs, **Backend-Devs**, **Artistas** y **Finanzas**. Los departamentos técnicos cuentan con dos grupos cada uno (**Empleados y Prácticas**) para poder aplicar políticas de acceso diferenciadas entre ambos perfiles. Artistas y Finanzas tienen un único grupo al ser perfiles homogéneos.

8.4. Traza de una petición extremo a extremo:

Cuando un usuario inicia sesión en el cliente Windows con sus credenciales de Active Directory y accede a **http://192.168.2.10**, el flujo es el siguiente:

- El cliente detecta que **192.168.2.10** no pertenece a su red (**192.168.1.0/24**) y envía el paquete a su gateway: el router Ubuntu en **192.168.1.1**.
- El router tiene habilitado **ip_forward** y una regla iptables que permite el reenvío entre enp0s3 (**red-empresa**) y enp0s8 (**red-servidores**), por lo que reenvía el paquete hacia **192.168.2.10**.
- El Ubuntu Server recibe la petición en el puerto 80, donde Docker tiene mapeado ese puerto al contenedor nginx.
- Nginx responde, el router devuelve la respuesta al cliente por la misma ruta, y el navegador muestra la página.

Las dos redes permanecen aisladas entre sí salvo por el reenvío explícito del router, simulando una arquitectura donde los servidores de producción no son directamente accesibles desde la red de usuarios.

9. Conclusiones y futuro:

9.1. Conclusiones:

GEKKINS ha resultado ser un proyecto técnicamente más ambicioso de lo previsto inicialmente. La combinación de un framework de escritorio poco maduro como Electrobus, la arquitectura multi-proceso con comunicación RPC y la gestión de infraestructura distribuida en tiempo real ha supuesto desafíos que van más allá del currículo habitual de ASIR.

Los dos retos técnicos más significativos han sido la gestión de estado concurrente y la adopción de la programación asíncrona. El sistema de estadísticas en tiempo real requiere mantener múltiples Maps sincronizados entre el worker de stats, el worker de autoescalado y los handlers RPC, todos ejecutándose de forma concurrente. Razonar sobre el estado compartido en un entorno asíncrono, qué datos están disponibles en cada momento, cuándo una caché puede estar desactualizada, cómo evitar condiciones de carrera, ha sido el aprendizaje más profundo del proyecto.

La elección de React como framework de interfaz ha funcionado correctamente, pero en retrospectiva Svelte hubiera sido una opción más natural para un perfil orientado al backend. React impone un modelo mental de componentes y reactividad que requiere tiempo para interiorizar, mientras que Svelte se aproxima más a HTML y JavaScript estándar, reduciendo la fricción para quien no tiene experiencia previa en frontend.

Por otro lado, haber construido la infraestructura manualmente, los túneles SSH, el autoescalado, el bootstrap de nodos, el sistema de conexión multi-nodo, ha sido una decisión deliberadamente formativa. Afrontar esos problemas sin abstracciones de alto nivel ha permitido entender en profundidad por qué herramientas como los agentes de infraestructura o Kubernetes existen y qué problemas resuelven. Esa comprensión no habría sido posible empezando directamente por la solución abstracta.

9.2. Trabajo futuro:

El desarrollo futuro de GEKKINS se plantea en tres fases diferenciadas:

Fase 1 -> MVP funcional. El objetivo inmediato es completar el MVP con el stack actual: desplegar el backend de la plataforma web con observabilidad real, implementar la internacionalización del frontend en español e inglés, y estabilizar las funcionalidades existentes de la app de escritorio. Esta fase sienta las bases para que GEKKINS sea una herramienta usable por terceros.

Fase 2 -> Migración tecnológica. Una vez consolidado el MVP, el plan es migrar el frontend web y la interfaz de la app de escritorio a SvelteKit, unificando front y back en un único proyecto y eliminando la separación artificial entre ambas capas que ha sido una fuente de complejidad innecesaria. Esta migración también implica adoptar Drizzle ORM para la gestión de la base de datos y rediseñar el sistema de autenticación con una solución más completa.

Fase 3 -> Evolución de la plataforma. La fase más ambiciosa implica tres cambios fundamentales en la arquitectura. **Primero**, migrar de Docker Swarm a Kubernetes como orquestador subyacente, manteniendo la abstracción visual que caracteriza a GEKKINS. **Segundo**, reemplazar el sistema de túneles SSH gestionados desde la app por agentes ligeros desarrollados en Go desplegados en cada nodo, lo que eliminaría la dependencia de que la app esté abierta para mantener la conectividad y permitiría una arquitectura verdaderamente distribuida. **Tercero**, implementar la configuración automática de redes privadas entre nodos, que es uno de los mayores puntos de fricción actuales para usuarios sin experiencia en redes.

La visión a largo plazo es que GEKKINS se convierta en la interfaz que hace que operar infraestructura distribuida deje de sentirse como mantenimiento y empiece a sentirse como construir producto, accesible para desarrolladores que quieren controlar su infraestructura sin convertirse en DevOps a tiempo completo.